

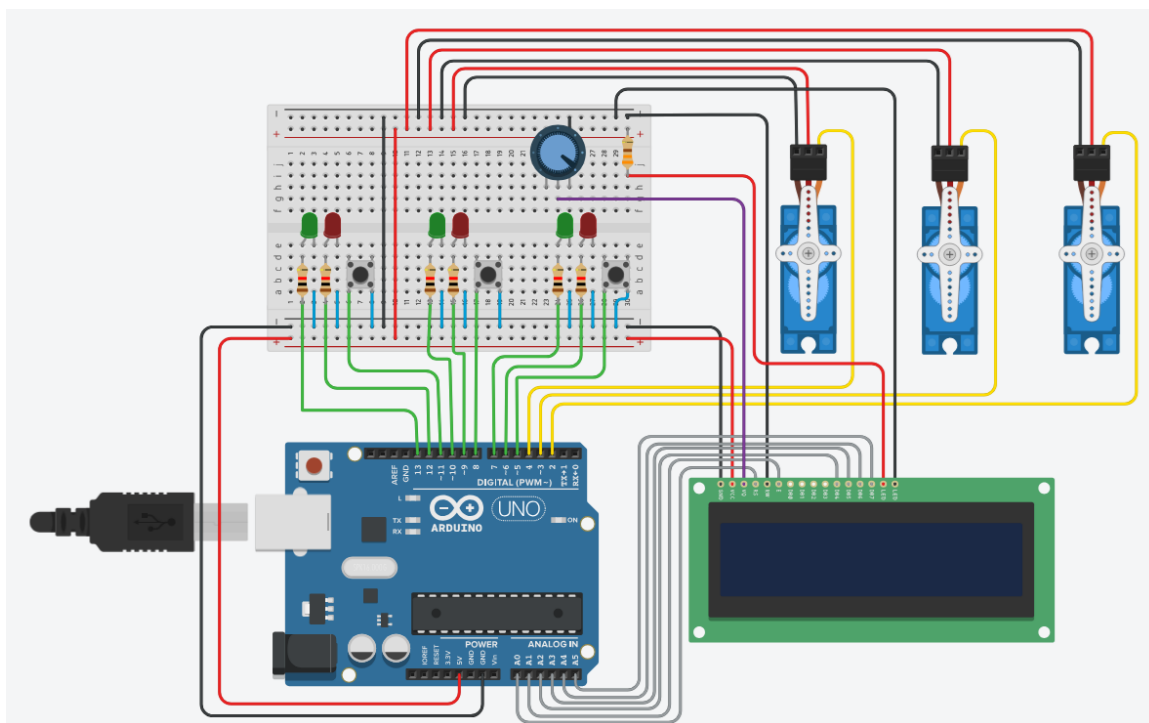
Neumann János Egyetem

Üzemmérnök-informatikus szak

Digitális technika II.

PROJEKTFELADAT DOKUMENTÁCIÓ

*Arduino-alapú csomagautomata modell
megvalósítása Tinkercad szimulációs környezetben*



Készítette:
Papp Péter

Neptun-kód:
FANSS

URL: <https://github.com/Pappeti/Csomagautomata>

Budapest, 2026.05.21

Tartalomjegyzék

Tartalomjegyzék	2
1. Bevezetés	3
2. A projekt feladata.....	3
3. Felhasznált eszközök és alkatrészek	4
4. A rendszer működési elve	5
5. Hardveres felépítés.....	6
6. Portkiosztás.....	8
7. A program működése	9
8. A program fontosabb részei	10
9. A projekt megvalósítása a Tincercad környezetben	11
10. A projekt megvalósítása a Tincercad környezetben	13
11. Összegzés	14
12. Továbbfejlesztési lehetőségek.....	15

1. Bevezetés

Napjainkban az automatizált rendszerek egyre nagyobb szerepet kapnak a mindennapi életben. Az egyik ilyen gyakorlati alkalmazási terület a csomagautomaták használata, amelyek lehetővé teszik a küldemények gyors, egyszerű és személyes átvétel nélküli kezelését. Ezek a rendszerek különösen hasznosak az online vásárlások elterjedésével, mivel a felhasználók rugalmas időpontban vehetik át csomagjaikat.

A csomagautomata működésének alapja, hogy több, egymástól elkülönített tárolórekeszből áll, amelyek nyitását és zárását valamilyen elektronikus vezérlőrendszer irányítja. A rendszer feladata, hogy jelezze az egyes rekeszek állapotát, kezelje a felhasználói műveleteket, és biztosítsa a megfelelő rekesz megnyitását. Egy ilyen rendszer megvalósítása jó példája annak, hogyan lehet szenzorok, kijelzők, vezérlőelemek és programozás segítségével automatizált folyamatokat létrehozni.

A projekt célja egy Arduino-alapú csomagautomata modell megtervezése és szimulációs környezetben történő megvalósítása volt. A megvalósítás során a Tinkercad alkalmazást használtam, amely lehetőséget biztosít elektronikai kapcsolások összeállítására és a program működésének tesztelésére. A modell három „rekeszt” tartalmaz, amelyekhez külön LED-es állapotjelzés, nyomógombos vezérlés és szervomotoros ajtónyitás tartozik. A rendszer működését egy LCD kijelző is támogatja, amely folyamatos visszajelzést ad a felhasználónak.

A választott projekt jól szemlélteti a digitális technika, a mikrokontrolleres vezérlés és az automatizálás alapelveit. A feladat során egyszerre kellett figyelembe venni a hardveres bekötést, az egyes alkatrészek összehangolt működését, valamint a vezérlőprogram logikai felépítését. Ezáltal a projekt nemcsak látványos, hanem műszaki szempontból is jól bemutatható megoldást eredményezett.

2. A projekt feladata

A projekt feladata egy olyan Arduino-alapú csomagautomata modell megtervezése és megvalósítása volt, amely képes több különálló rekesz állapotának kezelésére, a felhasználói vezérlés fogadására, valamint az egyes rekeszek nyitására és zárására szimulálására.

A feladat során három különálló rekesz működését kellett modellezni. Minden rekeszhez tartozik egy nyomógomb, amely a kiválasztást szolgálja, két LED, amelyek az adott rekesz állapotát jelzik, valamint egy szervomotor, amely az ajtó nyitását és zárását szimulálja. A rekesz foglaltsági állapotát a LED-ek mutatják: foglalt állapot esetén a piros LED világít, üres állapot esetén pedig a zöld LED. Ez lehetővé teszi, hogy a felhasználó azonnal lássa, mely rekeszek tartalmazznak csomagot, és melyek szabadok.

A rendszer működéséhez egy LCD kijelző is tartozik, amely szöveges visszajelzést ad a felhasználónak. Az LCD feladata többek között az induló üdvözlőképernyő megjelenítése, a rekesznyitás állapotának kijelzése, valamint annak jelzése, ha egy rekesz már üres. A kijelző használata azért fontos, mert így a rendszer nemcsak elektromos jelekkel és mozgással, hanem jól értelmezhető szöveges üzenetekkel is kommunikál.

A vezérlőprogramnak kezelnie kell a rendszer indulását, az összes alkatrész inicializálását, a rekeszek kiindulási állapotát, valamint a gombnyomások figyelését. A működés lényege, hogy a felhasználó egy adott rekeszhez tartozó gomb megnyomásával kezdeményezi a nyitást. Ha az adott rekesz foglalt, akkor a hozzá tartozó szervomotor kinyitja az ajtót, majd rövid idő múlva visszazárja azt. Ezzel egy időben a program a rekesz állapotát üresre állítja, és a LED-eket is ennek megfelelően frissíti. Ha a felhasználó olyan rekeszt próbál megnyitni, amely már üres, akkor a rendszer ezt az LCD kijelzőn jelzi.

A projekt fontos része volt az is, hogy a rendszer működése áttekinthető, logikus és jól bemutatható legyen. Emiatt a megoldás célja nem pusztán az egyes alkatrészek külön-külön történő működtetése volt, hanem egy összehangolt, valós problémát modellező rendszer létrehozása. A feladat tehát egyszerre foglalta magában a hardveres tervezést, a bekötések elkészítését, a szimulációs környezet használatát és a vezérlőprogram megírását.

3. Felhasznált eszközök és alkatrészek

A projekt megvalósításához több elektronikai alkatrészt és egy szimulációs környezetre volt szükség. Az egyes elemek kiválasztásánál fontos szempont volt, hogy a csomagautomata működésének fő részei jól modellezhetők legyenek, ugyanakkor az áramkör Tinkercadben is egyszerűen felépíthető maradjon. A rendszer központi eleme az Arduino Uno mikrokontrolleres fejlesztőpanel, amely az összes bemenet és kimenet kezelését végzi, illetve a vezérlőprogram futtatásáért felel.

A projektben három nyomógomb került felhasználásra, amelyek az egyes rekeszek kiválasztását teszik lehetővé. A felhasználó ezek segítségével tudja kezdeményezni a megfelelő rekesz nyitását. Minden rekeszhez két LED tartozik: egy piros és egy zöld. A piros LED azt jelzi, hogy az adott rekesz foglalt, vagyis csomag van benne, míg a zöld LED azt mutatja, hogy a rekesz üres. A LED-ek alkalmazása egyszerű és jól látható állapot-visszajelzést biztosít.

A rekeszek ajtajának nyitását és zárását három darab szervomotor szimulálja. A servók azért alkalmasak erre a feladatra, mert meghatározott szögbe forgathatók, így jól modellezhető velük az ajtók mozgása. A projektben minden rekeszhez egy külön szervo tartozik, amely a megfelelő gomb megnyomásakor működésbe lép. A szervomotorok vezérlését az Arduino digitális kimenetei végzik.

A rendszerhez egy 16x2 karakteres LCD kijelző is kapcsolódik, amely a felhasználó számára szöveges információkat jelenít meg. A kijelző például üdvözlő üzenetet mutat induláskor, illetve tájékoztatást ad arról, hogy melyik rekesz nyílik, záródik, vagy éppen üres. Az LCD kijelző kontrasztjának beállításához egy potméter is szükséges, amely lehetővé teszi a karakterek jól olvasható megjelenítését.

Az áramkör felépítéséhez kis szerelőlapot, összekötő vezetékeket használtam. A szerelőlap előnye, hogy forrasztás nélkül lehet rajta áramkört összeállítani, így a kapcsolat könnyen módosítható és áttekinthető marad. A LED-ek biztonságos használatához megfelelő értékű előtétellenállásokra is szükség volt, amelyek korlátozzák az áramerősséget, és megvédik a LED-eket a károsodástól. Az LCD háttérvilágításához szintén alkalmazni kellett áramkorlátozó ellenállást.

A projekt megvalósítása a Tinkercad online szimulációs környezetben történt. Ez az alkalmazás lehetőséget biztosít arra, hogy az elektronikai kapcsolásokat virtuálisan összeállítsuk, majd az Arduino programot közvetlenül a rendszerben futtassuk. Ennek előnye, hogy a hibák könnyebben felismerhetők, a működés ellenőrizhető, és a teljes rendszer valós eszközök nélkül is bemutatható.

A projekt során felhasznált főbb eszközök és alkatrészek tehát a következők voltak: Arduino Uno, kis szerelőlap, 16x2 LCD kijelző, potméter, három darab szervomotor, három darab nyomógomb, hat darab LED, ellenállások, összekötő vezetékek, valamint maga a Tinkercad szimulációs környezet.

4. A rendszer működési elve

A megvalósított csomagautomata modell működése azon alapul, hogy a rendszer három különálló rekeszt kezel egymástól függetlenül, mégis egy közös vezérlőegység, az Arduino Uno irányításával. Minden rekeszhez tartozik egy nyomógomb, két állapotjelző LED és egy szervomotor. Ezek együttesen biztosítják, hogy a felhasználó kiválaszthassa a kívánt rekeszt, visszajelzést kapjon annak állapotáról, majd szükség esetén a rekesz nyitása és zárása is végbemenjen.

A rendszer induláskor az Arduino inicializálja a bemeneteket és kimeneteket, valamint beállítja a rekeszek kezdeti állapotát. A projekt jelenlegi változatában a rekeszek induláskor foglaltnak számítanak, tehát a rendszer azt feltételezi, hogy mindegyikben csomag található. Ennek megfelelően a piros LED-ek világítanak, a zöld LED-ek pedig kikapcsolt állapotban maradnak. A szervomotorok kiindulási helyzetben zárt állapotot jelképeznek. Ezzel egy időben az LCD kijelzőn megjelenik egy rövid üdvözlő üzenet, majd a rendszer átvált a normál kezdőképernyőre.

A felhasználó a rekeszekhez tartozó nyomógombok segítségével tud interakcióba lépni a rendszerrel. Ha valamelyik gombot megnyomja, az Arduino ellenőrzi, hogy a kiválasztott

rekesz foglalt vagy üres állapotban van-e. Ha a rekesz foglalt, akkor a rendszer megkezdí a nyitási folyamatot. Ekkor az LCD kijelző szövegesen jelzi, hogy melyik rekesz nyílik, a hozzá tartozó szervomotor pedig elfordul, ezzel szimulálva az ajtó kinyitását. Néhány másodperc elteltével a szervomotor visszatér eredeti helyzetébe, így az ajtó bezáródik.

A nyitás és zárás végrehajtása után a program módosítja az adott rekesz állapotát. A korábban foglaltnak jelölt rekesz ezután üresnek számít, mivel a modell szerint a csomagot átvették. A LED-ek állapota ennek megfelelően frissül: a piros LED kikapcsol, a zöld LED pedig felkapcsol. Az LCD kijelző rövid ideig tájékoztatást ad arról, hogy a rekesz bezárult és most már üres, majd visszatér a kezdőképernyőre.

Ha a felhasználó olyan rekeszt próbál kiválasztani, amely már üres, akkor a rendszer nem hajt végre nyitást. Ebben az esetben az LCD kijelző jelzi, hogy a kiválasztott rekesz üres, vagyis abban már nincs átvehető csomag. Ez a működés megakadályozza a felesleges ajtómozgást, és egyértelmű visszajelzést ad a felhasználó számára a rendszer aktuális állapotáról.

A csomagautomata működésének egyik fontos eleme a különböző hardverelemek összehangolt együttműködése. A nyomógombok bemenetként szolgálnak, a LED-ek és a szervomotorok kimeneti elemek, az LCD pedig a felhasználói kommunikációt segíti. Az Arduino ezekből az információkból döntéseket hoz, és ennek megfelelően vezérli az egyes elemeket. Így a projekt jól szemlélteti egy egyszerű automatizált vezérlőrendszer működését, ahol a bemenetek, az állapotkezelés és a kimenetek szorosan összekapcsolódnak.

A rendszer működési elve tehát összefoglalva az, hogy a felhasználói gombnyomás hatására az Arduino kiértékeli a kiválasztott rekesz állapotát, szükség esetén kinyitja és bezárja azt, majd frissíti a rekesz állapotát és a hozzá tartozó vizuális visszajelzéseket. A folyamat minden lépése az LCD kijelzőn is követhető, ezáltal a modell működése jól érthetővé és szemléletessé válik.

5. Hardveres felépítés

A projekt hardveres felépítésének alapját az Arduino Uno fejlesztőpanel képezi, amely a rendszer központi vezérlőegységként működik. Az Arduino feladata az egyes bemenetek és kimenetek kezelése, a gombnyomások érzékelése, a LED-ek vezérlése, a szervomotorok működtetése, valamint az LCD kijelzőn megjelenő információk kezelése. A kapcsolás összeállítása kis szerelőlap segítségével történt, amely lehetővé tette az alkatrészek egyszerű, forrasztás nélküli összekapcsolását.

A tápellátás kialakításánál az Arduino 5 V-os és GND kimeneteit vezettem ki a szerelőlap tápsínjeire. Ez biztosítja, hogy a rendszer összes eleme közös tápfeszültségről működjön. A szerelőlap használata előnyös volt, mert így az egyes alkatrészek tápellátása átláthatóan és könnyen szervezhető módon valósulhatott meg. A pozitív és negatív tápvonalak

elosztása különösen fontos volt a LED-ek, a szervomotorok és az LCD kijelző megfelelő működéséhez.

A rendszer három rekeszt modellez, ezért minden rekeszhez külön hardverelemek tartoznak. Az egyes rekeszek állapotjelzését két LED biztosítja. A piros LED azt mutatja, hogy a rekesz foglalt, tehát csomagot tartalmaz, míg a zöld LED az üres állapotot jelzi. A LED-ek Arduino digitális kimenetekre csatlakoznak, és mindegyikhez előtétellenállás tartozik, amely az áramerősség korlátozására szolgál. Ez megvédi a LED-eket a túlterheléstől, és biztosítja a biztonságos működést.

A felhasználói vezérlést három darab nyomógomb biztosítja. Mindegyik gomb egy adott rekeszhez tartozik, és annak kiválasztását teszi lehetővé. A gombok bekötése úgy történt, hogy az Arduino digitális bemeneteihez csatlakoznak, a program pedig belső felhúzóellenállásos üzemmódban kezeli őket. Ennek előnye, hogy külső felhúzóellenállás alkalmazása nélkül is stabil állapotot lehet biztosítani, a gomb megnyomásakor pedig az adott bemenet alacsony logikai szintre vált.

Az egyes rekeszek ajtajának nyitását és zárását három darab szervomotor szimulálja. A szervók három vezetéssel csatlakoznak a rendszerhez: tápfeszültség, föld és jelvezeték. A jelvezetékek az Arduino digitális kimeneteire kapcsolódnak, míg a táp- és földvezetékek a szerelőlap megfelelő tápsínjeire kerültek. A szervomotorok használata azért előnyös, mert meghatározott szögbe állíthatók, így egyszerűen modellezhető velük a rekeszek ajtajának nyitási és zárási mozgása.

A szöveges visszajelzéshez egy 16x2 karakteres LCD kijelző került felhasználásra. A kijelző több adat- és vezérlőlábon keresztül kapcsolódik az Arduinohoz, és a rendszer állapotával kapcsolatos információkat jeleníti meg. Ilyen információ például az induló üdvözlőszöveg, a kiválasztott rekesz állapota, a nyitás és zárás folyamata, valamint az üres rekeszek jelzése. Az LCD kijelző kontrasztjának szabályozásához egy potmétert kellett alkalmazni, amely lehetővé tette a karakterek megfelelő láthatóságának beállítását.

A kijelző háttérvilágításának bekötésénél külön figyelmet kellett fordítani az áramkorlátozásra. A háttérvilágítás LED-jéhez ellenállást kellett kapcsolni, mivel közvetlen tápfeszültségre kötve a kijelző hibás működést jelezett. Ennek javítása után az LCD stabilan működött, és a szövegek megfelelően jelentek meg a kijelzőn. Ez a tapasztalat rámutatott arra, hogy a hardveres tervezés során a kisebb részletek is jelentősen befolyásolhatják a rendszer működését.

A hardveres felépítés kialakításánál fontos szempont volt az átláthatóság és a logikus elrendezés. A szerelőlapon az alkatrészek csoportosítva helyezkednek el: a gombok, a LED-ek, a szervók és az LCD kijelző külön funkcionális egységeket alkotnak. Ez nemcsak a bekötést könnyítette meg, hanem a hibakeresést és a rendszer működésének bemutatását is egyszerűbbé tette. Összességében a hardveres felépítés jól szemlélteti, hogyan kapcsolhatók össze különböző elektronikai elemek egy közös, mikrokontrollerrel vezérelt automatizált rendszerben.

6. Portkiosztás

A rendszer megfelelő működéséhez pontosan meg kellett határozni, hogy az egyes alkatrészek az Arduino Uno mely lábaira csatlakoznak. A portok kiosztásának kialakításánál fontos szempont volt, hogy minden bemenet és kimenet egyértelműen elkülöníthető legyen, valamint a bekötés logikus és áttekinthető maradjon. A projekt végleges változatában a digitális lábak szolgálnak a LED-ek, a nyomógombok és a szervomotorok vezérlésére, míg az analóg lábak az LCD kijelző kapcsolatait kezelik digitális funkcióban.

Az első rekeszhez tartozó zöld LED az Arduino D13 lábára, a piros LED pedig a D12 lábára csatlakozik. Az első rekesz kiválasztására szolgáló nyomógomb a D11 lábra került. A második rekesz esetében a zöld LED a D10, a piros LED a D9, a nyomógomb pedig a D8 lábára csatlakozik. A harmadik rekeszhez tartozó zöld LED a D7, a piros LED a D6 lábon található, míg a hozzá tartozó nyomógomb a D5 lábat használja.

A három szervomotor jelvezetékei szintén digitális lábakra kapcsolódnak. Az első rekeszhez tartozó szervomotor jelvezetéke a D4 lábra csatlakozik, a második rekesz szervója a D3 lábat használja, míg a harmadik rekesz szervója a D2 lábra kapcsolódik. A szervók tápellátása külön a szerelőlap tápsínjeiről történik, de a vezérlőjel minden esetben az Arduino megfelelő digitális kimenetéről érkezik.

Az LCD kijelző bekötése az Arduino analóg lábain keresztül valósult meg. Bár ezek a csatlakozók alapvetően analóg bemenetként is használhatók, a projektben digitális funkciót látnak el. Az LCD RS lába az A0, az E lába az A1, míg az adatátviteli vonalak közül a D4, D5, D6 és D7 rendre az A2, A3, A4 és A5 lábakra csatlakoznak. Ez a megoldás lehetővé tette, hogy az összes szükséges alkatrész kényelmesen elférjen az Arduino rendelkezésre álló portjain.

A portkiosztás megtervezésénél külön figyelmet kellett fordítani arra, hogy a D0 és D1 lábak ne kerüljenek felhasználásra. Ennek oka, hogy ezek a soros kommunikációért felelős RX és TX lábak, amelyeket az Arduino a program feltöltésekor és a számítógéppel való kommunikáció során használ. Ezek mellőzése stabilabb működést és egyszerűbb hibakeresést tett lehetővé. A végleges portkiosztás így teljes egészében a D2–D13 és az A0–A5 lábakra épül.

A portkiosztás összefoglalva a következő: D13, D12 és D11 az első rekeszhez tartoznak, D10, D9 és D8 a második rekeszt kezelik, D7, D6 és D5 a harmadik rekesz LED-jeit és gombját szolgálják ki, míg D4, D3 és D2 a három szervomotor vezérlését végzik. Az LCD kijelző az A0–A5 lábakon keresztül csatlakozik az Arduinohoz. Ez az elrendezés jól elkülöníti a funkciókat, és a rendszer teljes működését egy átlátható, logikus portstruktúrára építi.

7. A program működése

A csomagautomata működését egy Arduino nyelven megírt vezérlőprogram biztosítja, amely folyamatosan figyeli a nyomógombok állapotát, kezeli a rekeszek foglaltsági adatait, vezérli a LED-eket és a szervomotorokat, valamint üzeneteket jelenít meg az LCD kijelzőn. A program feladata tehát az, hogy a hardverelemek működését összehangolja, és biztosítsa a rendszer logikus, átlátható működését.

A program indulásakor először az összes szükséges könyvtár betöltése történik meg. A szervomotorok kezeléséhez a Servo könyvtárat, az LCD kijelző vezérléséhez pedig a LiquidCrystal könyvtárat használja a program. Ezek után következik a fontosabb változók, tömbök és állandók deklarálása. A program külön tömbökben tárolja az egyes rekeszekhez tartozó LED-ek, gombok és szervók lábkiosztását, ami jelentősen egyszerűsíti a későbbi vezérlést. A rekeszek állapotát egy logikai tömb tárolja, amely azt jelzi, hogy az adott rekesz foglalt vagy üres.

A setup() függvény a rendszer inicializálásáért felelős. Ebben a részben történik meg az LCD kijelző indítása, a LED-ek és gombok lábainak beállítása, valamint a szervók csatlakoztatása és induló pozícióba állítása. A gombok belső felhúzóellenállással működnek, ezért a program INPUT_PULLUP módban kezeli őket. Az inicializálás után a program frissíti a LED-eket az aktuális rekeszállapotoknak megfelelően, majd megjeleníti az induló üdvözlőképernyőt, végül átvált a normál kezdőképernyőre.

A program egyik fontos része a LED-ek állapotának kezelése. Erre egy külön függvény szolgál, amely minden rekesznél ellenőrzi az állapotváltozót, és ennek alapján dönti el, hogy a piros vagy a zöld LED világítson. Ha a rekesz foglalt, akkor a piros LED világít, ha pedig üres, akkor a zöld. Ez a megoldás biztosítja, hogy a LED-ek mindig összhangban maradjanak a programban tárolt logikai állapottal.

Az LCD kijelző kezelésére szintén külön függvények szolgálnak. Ezek gondoskodnak az induló üdvözlőszöveg, a kezdőképernyő, a rekesznyitás, a rekesz zárása, illetve az üres rekeszhez tartozó figyelmeztető üzenetek megjelenítéséről. A program úgy van kialakítva, hogy az LCD sorait minden új üzenet előtt teljesen felülírja, így elkerülhető a korábbi karakterek kijelzőn maradása. Ez különösen fontos volt a fejlesztés során, mivel a szimuláció elején előfordultak megjelenítési hibák.

A loop() függvény folyamatosan figyeli a három nyomógomb állapotát. Ha a felhasználó valamelyik gombot megnyomja, a program azonosítja, hogy melyik rekeszhez tartozik a kérés, majd ellenőrzi annak aktuális állapotát. Ha a rekesz foglalt, akkor meghívja a rekesznyitó függvényt. Ha a rekesz már üres, akkor a rendszer nem hajt végre nyitást, hanem az LCD kijelzőn erről ad tájékoztatást. A program megvárja a gomb felengedését is, ami megakadályozza, hogy egyetlen hosszabb gombnyomás többször egymás után váltson ki műveletet.

A rekesznyitó függvény végzi a szervomotor tényleges vezérlését. Először kijelzi az LCD-n, hogy melyik rekesz nyílik, majd a megfelelő szervót nyitott pozícióba forgatja. Néhány

másodpercnyi várakozás után a program bezárja a rekeszt, vagyis a szervót visszaállítja zárt helyzetbe. Ezt követően a rekesz állapotát üresre állítja, frissíti a LED-eket, majd az LCD kijelzőn megjeleníti, hogy a csomag átvétele megtörtént, és a rekesz immár üres. Ezután a rendszer visszatér a kezdőképernyőre, és készen áll a következő műveletre.

A program működésének fontos jellemzője, hogy egyszerű, mégis jól strukturált módon kezeli az egyes funkciókat. Az ismétlődő feladatokat külön függvényekbe szervezve valósítottam meg, így a kód áttekinthetőbbé és könnyebben karbantarthatóvá vált. A program logikája egyértelműen követi a rendszer működését: indulás, állapotjelzés, felhasználói beavatkozás, rekesznyitás, állapotfrissítés, majd visszatérés az alaphelyzetbe. Ennek köszönhetően a csomagautomata modell működése jól követhető, és a digitális vezérlés alapelvei szemléletesen bemutatathatók.

8. A program fontosabb részei

A vezérlőprogram több egymással együttműködő részből épül fel, amelyek közösen biztosítják a csomagautomata megfelelő működését. A program szerkezetének kialakításánál fontos szempont volt az áttekinthetőség és az, hogy az egyes feladatok jól elkülönüljenek egymástól. Ennek megfelelően a program elején található a könyvtárak, a változók és a lábkiosztások deklarációi, ezt követi az inicializálást végző `setup()` függvény, majd a folyamatos működésért felelős `loop()` függvény, valamint több kisebb segédfüggvény.

A program első fontos része a szükséges könyvtárak betöltése. A Servo könyvtár a szervomotorok vezérlését teszi lehetővé, míg a LiquidCrystal könyvtár az LCD kijelző kezeléséhez szükséges. Ezek nélkül a rendszer nem tudná végrehajtani sem a rekeszek nyitását és zárását, sem a szöveges üzenetek megjelenítését. A könyvtárak használata leegyszerűsíti a programozást, mivel az alacsonyabb szintű hardverkezelést nem kézzel kell megvalósítani.

A következő lényeges rész a változók, tömbök és konstansok meghatározása. A program tömbök segítségével kezeli az egyes rekeszekhez tartozó LED-eket, gombokat és szervókat. Ez a megoldás azért előnyös, mert a három rekesz azonos logika szerint működik, így ugyanaz a programrész használható mindegyikre. A rekeszek foglaltsági állapotát egy logikai tömb tárolja, amely minden rekeszhez egy igaz vagy hamis értéket rendel. Ezen kívül külön konstansok határozzák meg a szervomotorok nyitott és zárt pozícióját.

A `setup()` függvény a rendszer indulásakor egyszer fut le, és az összes szükséges kezdeti beállítást elvégzi. Ebben a részben történik meg az LCD kijelző inicializálása, a LED-ek kimeneti lábbá, valamint a gombok bemeneti lábbá állítása. Ugyanitt csatlakozik a három szervomotor a megfelelő vezérlőportokhoz, majd mindegyik zárt helyzetbe áll. A `setup()` feladata továbbá, hogy a program kezdő állapotát jól láthatóan megjelenítse: a LED-ek a

rekeszek aktuális foglaltságát mutatják, az LCD pedig először az üdvözlőképernyőt, majd a normál kezdőképet jeleníti meg.

A LED-ek vezérlését egy külön függvény végzi, amely minden rekesznél megvizsgálja az aktuális állapotot. Ennek alapján kapcsolja fel a megfelelő piros vagy zöld LED-et. Ennek a függvénynek nagy jelentősége van, mert biztosítja, hogy a felhasználó által látott állapotjelzés mindig megfeleljen a program által nyilvántartott rekeszállapotnak. Ha például egy rekeszből a csomagot átvették, a program azonnal frissíti a LED-ek állapotát is.

A program szempontjából fontos szerepet töltenek be az LCD kezelésére írt függvények is. Ezek felelnek az induló üdvözlés, a kezdőképernyő, a nyitási állapot, a zárási folyamat, illetve az üres rekeszhez kapcsolódó üzenetek megjelenítéséért. Az LCD-szövegek külön kezelése azért előnyös, mert a program így jobban strukturálható, és a felhasználói visszajelzés logikája elkülönül a hardvervezérlés logikájától. Ez könnyebbé teszi a későbbi módosítást és fejlesztést is.

A rekesznyitó függvény a rendszer egyik legfontosabb része. Ez hajtja végre azt a folyamatot, amely a kiválasztott rekeszhez tartozó szervomotor működtetését és a rekesz állapotának megváltoztatását végzi. A függvény először kijelzi, hogy melyik rekesz nyílik, majd a megfelelő szervót nyitott helyzetbe forgatja. Ezután egy rövid várakozás következik, amely a csomag átvételének idejét szimulálja. A várakozási idő letelte után a függvény bezárja a rekeszt, üresre állítja annak logikai állapotát, frissíti a LED-eket, majd újabb LCD-üzenettel tájékoztatja a felhasználót a folyamat befejezéséről.

A `loop()` függvény a program folyamatosan ismétlődő része, amely állandóan figyeli a gombok állapotát. Ha valamelyik gombot megnyomják, a program azonosítja a megfelelő rekeszt, majd eldönti, hogy kell-e nyitási folyamatot indítani. Ebben a részben történik a felhasználói beavatkozás kezelése, ezért ez a program működésének központi eleme. A `loop()` függvény gondoskodik arról is, hogy a gomb lenyomása után a program megvárja a felengedést, így elkerülhető az, hogy ugyanarra a gombnyomásra több művelet is lefusson.

Összességében a program fontosabb részei jól elkülöníthetők egymástól, és mindegyiknek megvan a maga szerepe a rendszer egészének működésében. A könyvtárak biztosítják a hardverelemek kezelését, a változók és tömbök tárolják az állapotokat és a lábkiosztásokat, a `setup()` létrehozza a kezdő működési állapotot, a `loop()` figyeli a felhasználói műveleteket, a segédfüggvények pedig a LED-ek, az LCD és a szervók összehangolt vezérlését végzik. Ez a felépítés jól szemlélteti egy egyszerű beágyazott rendszer programjának logikai szerkezetét.

9. A projekt megvalósítása a Tincercad környezetben

A projekt megvalósítása a Tinkercad online szimulációs környezetben történt, amely lehetőséget biztosít elektronikai áramkörök virtuális összeállítására, valamint Arduino-alapú programok tesztelésére. A Tinkercad használatának előnye, hogy a teljes rendszer fizikai alkatrészek nélkül is felépíthető és kipróbálható, így a hibák gyorsabban felismerhetők, a kapcsolások egyszerűbben módosíthatók, és a működés jól szemléltethető. A projekt szempontjából ez különösen hasznos volt, mivel a csomagautomata több eltérő hardverelem összehangolt működését igényelte.

A megvalósítás első lépéseként létrehoztam egy új áramkört a Tinkercad Circuits felületén. Ezt követően elhelyeztem az Arduino Uno panelt és a szerelőlapot, amelyek a rendszer alapját képezték. A szerelőlap lehetővé tette az alkatrészek rendezett, áttekinthető bekötését, míg az Arduino biztosította a vezérléshez szükséges bemeneti és kimeneti portokat. A tápellátás kiépítéséhez az Arduino 5 V-os és GND csatlakozásait vezettem ki a szerelőlap megfelelő tápsínjeire.

Ezután következett a rekeszekhez tartozó LED-ek és nyomógombok elhelyezése. Minden rekeszhez két LED került: egy piros és egy zöld, amelyek az adott rekesz foglalt, illetve üres állapotát jelzik. A LED-eket előtétellenállásokon keresztül csatlakoztattam az Arduino digitális portjaihoz. A nyomógombok szintén a szerelőlapra kerültek, és a megfelelő bemeneti lábakhoz lettek kapcsolva. A Tinkercad szimuláció lehetőséget adott arra, hogy a gombok működését külön is ellenőrizsem, ami segített a későbbi hibakeresés során.

A következő lépés a három szervomotor bekötése volt, amelyek a rekeszek ajtajának nyitását és zárását szimulálják. A szervók mindegyike három vezetékkel csatlakozik: tápfeszültséggel, földeléssel és vezérlőjellel. A táp- és földvezetékek a szerelőlap megfelelő sínjeire kerültek, míg a jelvezetékeket az Arduino kijelölt digitális portjaira kötöttem. A Tinkercadban a szervók mozgása jól követhető, ezért könnyen ellenőrizhető volt, hogy a program megfelelő szögbe állítja-e őket.

A rendszer fontos része volt a 16x2 karakteres LCD kijelző beépítése is. Az LCD több adat- és vezérlővezetékkel csatlakozik az Arduino analóg portjaihoz, amelyeket a projektben digitális funkcióra használtam. A kijelző megfelelő működéséhez potmétert is be kellett építeni, amely a kontraszt beállítását biztosította. A fejlesztés során az LCD háttérvilágításánál külön ellenállást kellett alkalmazni, mert ennek hiányában a Tinkercad túláramot jelzett, és a kijelzőn hibát mutatott. A probléma javítása után az LCD stabilan megjelenítette a kívánt szövegeket.

A hardveres kapcsolás elkészítése után következett a programkód beillesztése a Tinkercad kódszerkesztőjébe. A program szöveges nézetben került megírásra, majd a szimuláció futtatásával lehetett ellenőrizni a rendszer működését. A szimuláció során figyeltem a LED-ek állapotváltozását, a gombok hatását, a szervók mozgását és az LCD kijelzőn megjelenő üzeneteket. A Tinkercad különösen hasznosnak bizonyult abban, hogy azonnal láthatóvá tette a bekötési hibákat és a logikai problémákat.

A megvalósítás során több hibát is fel kellett deríteni és javítani. Az egyik legfontosabb probléma az LCD kijelző helytelen működése volt, amely részben a potméter beállításával, részben pedig a háttérvilágítás hibás bekötésével függött össze. Emellett a gombok

működését is külön tesztelni kellett, hogy eldönthető legyen, hardveres vagy programozási eredetű hibáról van-e szó. A Tinkercad lehetőséget adott arra, hogy ezeket a problémákat fokozatosan, biztonságosan lehessen javítani.

Összességében a Tinkercad környezet jól támogatta a projekt megvalósítását. Segítségével a teljes csomagautomata-modell felépíthető, programozható és működés közben is ellenőrizhető volt. A szimuláció nemcsak a megvalósítást könnyítette meg, hanem a rendszer működésének bemutatását is szemléletesebbé tette. Ezáltal a projekt jól alkalmas arra, hogy bemutassa egy egyszerű mikrokontrolleres vezérlőrendszer tervezésének és kivitelezésének főbb lépéseit.

10. A projekt megvalósítása a Tinkercad környezetben

A projekt megvalósítása során nemcsak a hardverelemek összeépítése és a program elkészítése jelentett feladatot, hanem a felmerülő hibák felismerése és javítása is. A fejlesztés egyik legfontosabb tapasztalata az volt, hogy egy összetettebb Arduino-rendszer esetében a hibák gyakran nem egyetlen okra vezethetők vissza, hanem a hardveres bekötés és a programozás együtt befolyásolja a működést. Emiatt a hibakeresést lépésről lépésre, az egyes részegységeket külön ellenőrizve kellett elvégezni.

Az első jelentősebb probléma az LCD kijelző működésével kapcsolatban jelentkezett. A szimuláció indításakor a kijelzőn a várt szöveg helyett hibás karakterek, illetve egy piros hibajelzés jelent meg. A vizsgálat során kiderült, hogy a kijelző ugyan kapott tápfeszültséget, de a kontraszt nem volt megfelelően beállítva, ezért a karakterek nem látszottak jól. Ezt a hibát a potméter állításával lehetett javítani, amely után az üdvözlőszöveg már megjelent a kijelzőn. Ez jól mutatta, hogy az LCD helyes működéséhez nem elegendő a megfelelő adatkapcsolat, hanem a kontraszt beállítása is elengedhetetlen.

A kijelzőn megjelenő piros hibajelzés másik oka az LCD háttérvilágításának hibás bekötése volt. A Tinkercad figyelmeztetése alapján túl nagy áram folyt át a háttérvilágítás LED-jén, mert annak tápellátása kezdetben nem kapott megfelelő áramkorlátozó ellenállást. A hiba javítása után az LCD stabilan működött, és a hibajelzés megszűnt. Ez a tapasztalat rámutatott arra, hogy még egy kisebbnek tűnő részlet, például a háttérvilágítás bekötése is komoly működési problémát okozhat, ha nem megfelelően van kialakítva.

A nyomógombokkal kapcsolatban szintén felmerült hiba. Egy időszakban a gombok megnyomásának nem volt látható hatása, ezért először nem volt egyértelmű, hogy hardveres vagy programozási probléma okozza-e a jelenséget. Ennek eldöntésére külön tesztprogramot kellett készíteni, amely kizárólag a gombok állapotát vizsgálta. A teszt alapján kiderült, hogy a gombok hardveresen helyesen voltak bekötve, tehát a probléma a főprogram gombkezelő logikájában keresendő. A kód egyszerűsítése és a gombnyomás kezelésének módosítása után a rendszer már helyesen reagált a felhasználói bemenetekre.

A szervomotorok működésével kapcsolatban is voltak megfigyelhető jelenségek. A szimuláció indításakor a szervók enyhén elmozdultak, ami elsőre hibának tűnhetett, valójában azonban a program által megadott induló pozíció felvételét jelentette. Ez azt mutatta, hogy a szervók helyesen kapták meg a vezérlőjelet, és a program által meghatározott zárt állapotba álltak be. A fejlesztés során így meg kellett tanulni megkülönböztetni a valódi hibákat azoktól a működési jelenségektől, amelyek a rendszer normál viselkedéséhez tartoznak.

A portkiosztás kialakítása során szintén fontos tervezési döntéseket kellett hozni. Felmerült a D0 és D1 lábak használata, azonban ezek az Arduino soros kommunikációs lábai, ezért alkalmazásuk későbbi problémákat okozhatott volna a programfeltöltés és a kommunikáció során. A végleges megoldásban ezért ezek a portok nem kerültek felhasználásra. Ez a döntés egyszerűbbé és stabilabbá tette a projektet, ugyanakkor rámutatott arra is, hogy a lábkiosztás megtervezése önmagában is fontos része egy mikrokontrolleres rendszer kialakításának.

A fejlesztés során fontos tapasztalat volt az is, hogy a Tinkercad szimulációs környezet jól támogatja a hibák felismerését. A rendszer figyelmeztetései, a komponensek állapotának vizuális megjelenítése, valamint a szimuláció közbeni közvetlen ellenőrizhetőség jelentősen megkönnyítették a hibakeresést. Ennek ellenére a hibák megoldása sok esetben nem automatikusan történt, hanem logikus végiggondolást, külön tesztelést és fokozatos javítást igényelt.

Összességében a projekt megvalósítása során szerzett tapasztalatok azt mutatták, hogy egy összetettebb Arduino-rendszer építése során a sikeres működéshez nem elegendő csupán az alkatrészek összekötése és a program megírása. Ugyanilyen fontos a hibák módszeres felderítése, az egyes részegységek külön tesztelése, valamint a hardver és a szoftver összehangolt vizsgálata. A hibakeresési folyamat végül hozzájárult ahhoz, hogy a rendszer stabilan működő, jól bemutatható projektfeladattá váljon.

11. Összegzés

A projekt célja egy Arduino-alapú csomagautomata modell megtervezése és megvalósítása volt Tinkercad szimulációs környezetben. A feladat során sikerült létrehozni egy olyan működő rendszert, amely három különálló rekeszt kezel, és ezek állapotát LED-ekkel, nyomógombokkal, szervomotorokkal és LCD kijelzővel szemlélteti. A modell jól bemutatja, hogyan lehet egy hétköznapi, valós problémát elektronikai és programozási eszközök segítségével egyszerűsített formában megvalósítani.

A megoldás legfontosabb eredménye, hogy a rendszer képes kezelni a rekeszek foglaltsági állapotát, figyelni a felhasználói beavatkozásokat, végrehajtani a rekeszek nyitását és zárását, valamint folyamatos visszajelzést adni a felhasználónak. A piros és zöld LED-ek egyértelműen mutatják az egyes rekeszek állapotát, a szervomotorok látványosan szimulálják az ajtók mozgását, az LCD kijelző pedig szöveges információkkal teszi még

érthetőbbé a működést. Ennek köszönhetően a projekt nemcsak műszakilag működőképes, hanem szemléltetésre is jól alkalmas.

A projekt elkészítése során több fontos tapasztalatot is szereztem. Megismerhetővé vált az Arduino-alapú rendszerek felépítése, a digitális bemenetek és kimenetek kezelése, a szervomotorok vezérlése, valamint az LCD kijelző használata. Emellett a feladat rámutatott arra is, hogy a sikeres megvalósításhoz a hardveres bekötés és a programozás összehangolt tervezése szükséges. A hibakeresési folyamatok során különösen jól láthatóvá vált, hogy még kisebb bekötési hibák vagy logikai problémák is jelentősen befolyásolhatják a rendszer működését.

A Tinkercad használata nagy segítséget jelentett a projekt során, mivel lehetővé tette az áramkör biztonságos és költségmentes kipróbálását. A szimuláció révén a hibák fokozatosan javíthatók voltak, és a rendszer működése könnyen nyomon követhetővé vált. Ez különösen előnyös volt a kijelző, a gombok és a servók viselkedésének ellenőrzésénél, valamint a végleges, stabil működés kialakításánál.

Összességében a projekt sikeresen teljesítette a kitűzött célokat. A megvalósított csomagautomata modell alkalmas arra, hogy bemutassa a mikrokontrolleres vezérlés, az állapotkezelés és az automatizált működés alapelveit. A rendszer egyszerűsített formában ugyan, de jól érzékelteti a valós csomagautomaták működési logikáját. A projekt egyúttal megfelelő alapot nyújt további fejlesztésekhez is, például kódos azonosítás, hangjelzés vagy adminisztrációs funkciók beépítéséhez.

12. Továbbfejlesztési lehetőségek

A megvalósított csomagautomata modell jelenlegi formájában sikeresen bemutatja az alapvető működési elvet, ugyanakkor több olyan irány is létezik, amely mentén a rendszer továbbfejleszthető lenne. Ezek a bővítések egyrészt valóságghűbbé tehetnék a működést, másrészt összetettebb vezérlési és programozási feladatokat is lehetővé tennének. A projekt így jó kiindulási alapot jelent további fejlesztésekhez és új funkciók beépítéséhez.

Az egyik legkézenfekvőbb továbbfejlesztési lehetőség a kódos azonosítás bevezetése lenne. Jelenleg a rekeszek a hozzájuk tartozó gomb megnyomásával közvetlenül nyithatók, azonban egy valós csomagautomata esetében a hozzáférés rendszerint valamilyen azonosítás után történik. Ennek modellezésére a rendszer kiegészíthető lenne például billentyűzettel vagy gombkombinációs kódbevitellel. Ebben az esetben a rekesz csak helyes kód megadása után nyílna ki, ami jelentősen növelné a modell valósághűségét.

További lehetőség a hangjelzés beépítése buzzer segítségével. A hangjelzés többféle célt is szolgálhatna: jelezhetné a sikeres rekesznyitást, a hibás műveletet, vagy akár figyelmeztetést is adhatna. Ez a funkció javítaná a rendszer felhasználói visszajelzéseit, és még életszerűbbé tenné a működést. A fejlesztés során ez a lehetőség felmerült, de a végleges változatban a rendszer egyszerűbb kialakítása érdekében nem került beépítésre.

A projekt bővíthető lenne adminisztrációs funkciókkal is. Egy ilyen fejlesztés lehetővé tenné például új csomagok „betöltését” a rekeszekbe, vagyis azt, hogy egy üres rekesz újra foglalttá váljon. Jelenleg a rendszer csak a csomag átvételét modellezi, tehát a rekesz állapota foglalttól üresre változik. Egy továbbfejlesztett változatban szükség lenne egy olyan műveletre is, amely újra csomagot helyez el a rekeszben. Ezzel a csomagautomata működési ciklusa teljesebbé válna.

Szintén továbbfejlesztési lehetőség a rekeszek számának növelése. A jelenlegi modell három rekeszt kezel, ami szemléltetésre elegendő, de egy bővített rendszer már több tárolóhelyet is kezelhetne. Ez természetesen a hardver és a programozás összetettségét is növelné, de jól mutatná, hogyan skálázható egy ilyen típusú automatizált rendszer. Ebben az esetben szükség lehetne több bemenetre és kimenetre, vagy akár külön kiegészítő áramkörök alkalmazására is.

A felhasználói felület is fejleszthető lenne. A jelenlegi LCD kijelző egyszerű, de jól használható szöveges visszajelzést biztosít. Egy további fejlesztési irány lehetne egy részletesebb menürendszer kialakítása, amely több információt jelenítené meg, illetve többféle állapotot kezelne. Például megjeleníthető lenne, hogy hány rekesz foglalt, mely rekeszek szabadok, vagy akár külön üzenetek jelenhetnének meg különböző hibás műveletek esetén.

A rendszer működése valós szenzorokkal is kiegészíthető lenne. Például minden rekeszbe kerülhetne egy érzékelő, amely azt figyel, hogy van-e ténylegesen csomag a rekeszben. Ez még pontosabb állapotkezelést tenne lehetővé, hiszen a foglaltság nemcsak programból, hanem valós érzékelés alapján is meghatározható lenne. Ezzel a rendszer közelebb kerülne a valódi ipari vagy logisztikai automaták működési elvéhez.

Összességében a projekt jelenlegi formájában egy jól működő alapmodellt valósít meg, ugyanakkor több irányban is továbbfejleszthető. A kódos azonosítás, a hangjelzés, az adminisztrációs funkciók, a több rekesz kezelése, az összetettebb kijelzés és a szenzorok alkalmazása mind olyan lehetőségek, amelyekkel a rendszer összetettebbé, valóságosabbá és szakmailag még erősebbé tehető. Ez azt is mutatja, hogy a projekt nemcsak önmagában állja meg a helyét, hanem jó alapot jelent további fejlesztésekhez is.